

Object Oriented Programming (OOP)

Lecture 9: Class Relationships: Association, Aggregation, and Composition





Table of Contents

- Association
- Aggregation
- Composition
- Abstraction



Types of Relationships in Object Oriented Programming (OOP)

- Inheritance "is a" relationship
- Association "has-a" relationship
 - Composition "part-of" relationship
 - Aggregation "has-a" relationship



Understanding "Has-A" Relationships in Java

Association, Aggregation, and Composition



What are these relationships? How to build them?

- If two classes need to communicate with each other, there must be a link between them, and that can be represented by an association (connector).
- At first this relationships can be represented in the unified class diagram (UML) as we used it previously to convert a class from just a diagram to code.



Class: Car

- name	: String
- maxSpeed	: Integer
- price	: Float
- model	: Integer
+ setName(:String)	: Void
+ getName()	: String
..	..
..	..
..	..

UML diagram

- When the UML diagram is built correctly, it becomes possible to automatically generate the corresponding code using tools such as Sparx Systems Enterprise Architect.
- **UML (Unified Modeling Language)** is a standard set of diagrams used by software developers to design, visualize, and document software systems.

List of UML Diagram Types

Structure Diagrams:

- Class Diagram
- Component Diagram
- Deployment Diagram
- Object Diagram
- Package Diagram
- Profile Diagram
- Composite Structure Diagram

Behavioral Diagrams:

- Use Case Diagram
- Activity Diagram
- State Machine Diagram
- Sequence Diagram
- Communication Diagram
- Interaction Overview Diagram
- Timing Diagram

What is a Reference?

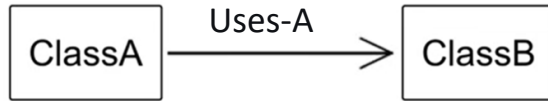
- Before understanding these relationships, we must understand what a "reference" is.
- A reference is a variable that stores the memory address of an object. It "points to" the object.

```
Car myCar = new Car("Toyota");  
//  ↑      ↑  
// Reference Object
```



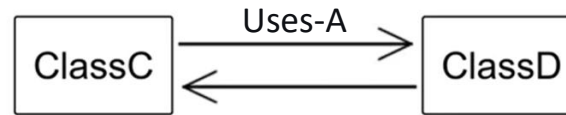
Association – The Weakest Relationship

- **Association** is a relationship where one class **uses** or **knows about** another class. Both objects can exist **independently**.



ClassA knows about ClassB

ClassB knows nothing about ClassA



ClassC knows about ClassD

ClassD knows about ClassC



Association – The Weakest Relationship

Code Pattern

```
public class ClassA {  
    private ClassB reference; // ← This is the reference  
  
    public void setReference(ClassB b) { // ← Object comes from OUTSIDE  
        this.reference = b;  
    }  
}
```



Association- Example (1/2)

```
// Car can exist alone  
public class Car {  
    private String model;  
    public Car(String model) { this.model = model; }  
    public String getModel() { return model; }  
}
```

```
// Driver USES Car (Association)  
public class Driver {  
    private String name;  
    private Car car; // Single reference  
  
    public Driver(String name) { this.name = name; }  
  
    public void assignCar(Car car) { this.car = car; }  
  
    public void drive() {  
        if (car != null)  
            System.out.println(name + " drives " + car.getModel());  
    }  
}
```

Association- Example (2/2)

- Driver and Car are completely independent.
- Both can exist without the other. The driver simply "uses" the car.

```
public class Main {  
    public static void main(String[] args) {  
        Car toyota = new Car("Toyota");  
        Driver ahmed = new Driver("Ahmed");  
  
        ahmed.assignCar(toyota); // Association  
        ahmed.drive();  
  
        // Driver can lose car – both still exist  
        ahmed.assignCar(null);  
        System.out.println("Car still exists: " + toyota.getModel());  
    }  
}
```

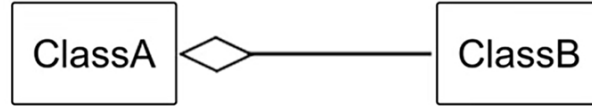


Aggregation

- Aggregation is a "has-a" relationship where the whole contains parts, but the parts can exist independently of the whole.

Aggregation "has-a" relationship

Aggregation weak relationship



ClassB is part of ClassA



Aggregation Vs. Association

Feature	Association	Aggregation
Simple Meaning	"Uses-A"	"Has-A" (weak)
Independence	Objects are completely independent	Parts can exist without the whole
Ownership	No ownership	Weak ownership
Lifespan	Different lifecycles	Part outlives whole
Code Pattern	Single reference	Array or collection
Example	Driver → Car	Garage → Car



Aggregation

Code Pattern

```
public class Whole {  
    private Part[] parts; // ← Array of references  
    private int count;  
  
    public void addPart(Part p) { // ← Parts come from OUTSIDE  
        parts[count] = p;  
        count++;  
    }  
}
```



Aggregation- Example (1/2)

// Car can exist alone (same as before)

```
public class Car {  
    private String model;  
    public Car(String model) { this.model = model; }  
    public String getModel() { return model; }  
}
```

// Garage HAS Cars (Aggregation)

```
public class Garage {  
    private String name;  
    private Car[] cars; // Array of references  
    private int count;  
  
    public Garage(String name, int capacity) {  
        this.name = name;  
        this.cars = new Car[capacity];  
        this.count = 0;  
    }  
  
    public void addCar(Car car) { // Car comes from outside  
        if (count < cars.length) {  
            cars[count++] = car;  
            System.out.println(car.getModel() + " added to " + name);  
        }  
    }  
  
    public void display() {  
        System.out.println(name + " has " + count + " cars:");  
        for (int i = 0; i < count; i++) {  
            System.out.println("  - " + cars[i].getModel());  
        }  
    }  
}
```

Aggregation- Example (2/2)

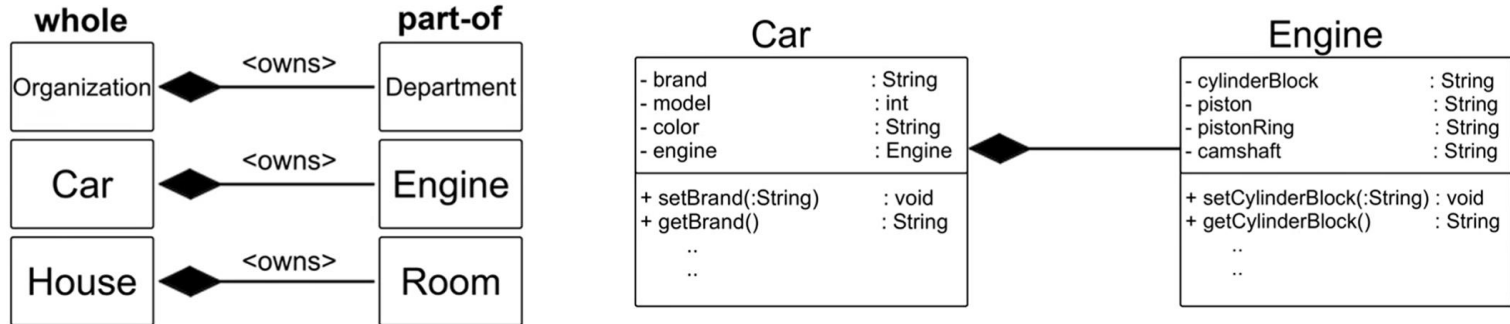
Aggregation = "Has-A"

(whole contains parts, but parts can live without the whole).

```
public class Main {  
    public static void main(String[] args) {  
        Car car1 = new Car("Toyota");  
        Car car2 = new Car("Honda");  
  
        Garage garage = new Garage("Downtown Garage", 5);  
  
        garage.addCar(car1); // Aggregation  
        garage.addCar(car2);  
        garage.display();  
  
        // Garage destroyed – Cars still exist  
        garage = null;  
        System.out.println("Cars still exist: " + car1.getModel() + ", " + car2.get  
Model());  
    }  
}
```

Composition

- Composition is a strong "has-a" relationship where the part cannot exist independently of the whole. If the whole is destroyed, the parts are destroyed too.



Aggregation Vs. Association Vs. Composition

Relationship	Simple Meaning	Code	Example
Association	Uses	Driver → Car	Driver uses Car
Aggregation	Has (parts can leave)	Garage → Car[]	Garage has Cars
Composition	Owns (parts die with it)	Car → Engine	Car owns Engine



Composition

Code Pattern

```
public class Whole {  
    private Part part; // ← Reference to Part  
  
    public Whole() {  
        this.part = new Part(); // ← Part created INSIDE (key difference!)  
    }  
}
```



Composition- Example (1/2)

```
// Engine cannot live alone
class Engine {
    String type;
    Engine(String type) {
        this.type = type;
        System.out.println("Engine created: " + type);
    }
    void start() { System.out.println("Engine starting...");
    }
}
```

```
// Car OWNS Engine
class Car {
    String model;
    Engine engine; // Car OWNS Engine

    Car(String model, String engineType) {
        this.model = model;
        this.engine = new Engine(engineType); // Engine created INSIDE
        System.out.println("Car created: " + model);
    }

    void start() {
        System.out.print(model + " starts: ");
        engine.start();
    }
}
```

Composition- Example (2/2)

- Car "owns" its Engine and Wheels. They are created inside the Car and cannot exist without it.
- When the Car is destroyed, all its parts are destroyed too.

```
// Test
public class Main {
    public static void main(String[] args) {

        Car myCar = new Car("Toyota", "V6");
        myCar.start();

        // Engine dies when car dies
        myCar = null;
        System.out.println("Car destroyed → Engine also destroyed");
    }
}
```



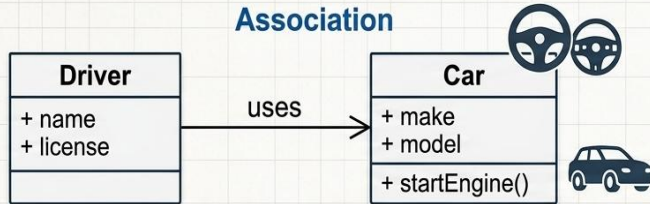
Conclusion of this part

ASSOCIATION: Driver → Car (Uses, outside)

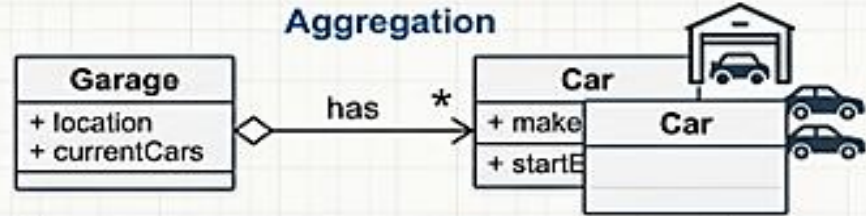
AGGREGATION: Garage → Car[] (Has many, outside)

COMPOSITION: Car → Engine (Owns, inside)

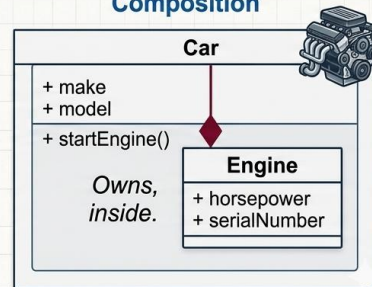
Association



Aggregation



Composition



Abstraction Concept



What is Abstraction?

- Abstraction is the OOP concept of hiding complex implementation details and exposing only the essential features of an object.
- Show WHAT an object does. Hide HOW it does it.

The Core Idea

What the User Sees	What the User Does Not See
Simple interface	Complex internal logic
Easy-to-use methods	How those methods work
Clear purpose	Implementation details



What is Abstraction?

Bad Code – No Abstraction

```
// BAD: User needs to know all internal steps
class Car {
    void startEngine() { System.out.println("Engine started"); }
    void checkFuel() { System.out.println("Fuel OK"); }
    void engageGear() { System.out.println("Gear engaged"); }
    void releaseBrake() { System.out.println("Brake released"); }
}

// User must do EVERY step manually
public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.startEngine(); // Step 1
        myCar.checkFuel(); // Step 2
        myCar.engageGear(); // Step 3
        myCar.releaseBrake(); // Step 4
        System.out.println("Car is moving");
    }
}
```

What is Abstraction?

Problems of the previous code

Problem	Why Bad
Too many steps	User must call 4 methods
User needs internal knowledge	Must know order of steps
Easy to forget a step	Car won't work properly
Hard to use	Complicated interface



What is Abstraction?

// GOOD: One simple method hides all complexity

```
class Car {  
    // Hidden methods (user never sees)  
    private void startEngine() { System.out.println("Engine started"); }  
    private void checkFuel() { System.out.println("Fuel OK"); }  
    private void engageGear() { System.out.println("Gear engaged"); }  
    private void releaseBrake() { System.out.println("Brake released"); }  
    // ONE simple method – user sees ONLY this  
    public void drive() {  
        startEngine(); // Automatic  
        checkFuel(); // Automatic  
        engageGear(); // Automatic  
        releaseBrake(); // Automatic  
        System.out.println("Car is moving");  
    }  
}
```

Good Code (With Abstraction)

// User does ONE simple step

```
public class Main {  
    public static void main(String[] args) {  
        Car myCar = new Car();  
        myCar.drive(); // One method – all steps happen automatically  
    }  
}
```



What is Abstraction?

Benefits of the previous code

Benefit	Why Good
One step	User calls only drive()
No internal knowledge needed	User doesn't need to know steps
Cannot forget steps	All steps happen automatically
Easy to use	Simple, clean interface



Any Question?

